

```

1)
public String agendaDoDia(LocalDate data){
    if(data == null)
        throw new ArgumentNullException("Data não pode ser nula para a agenda.");

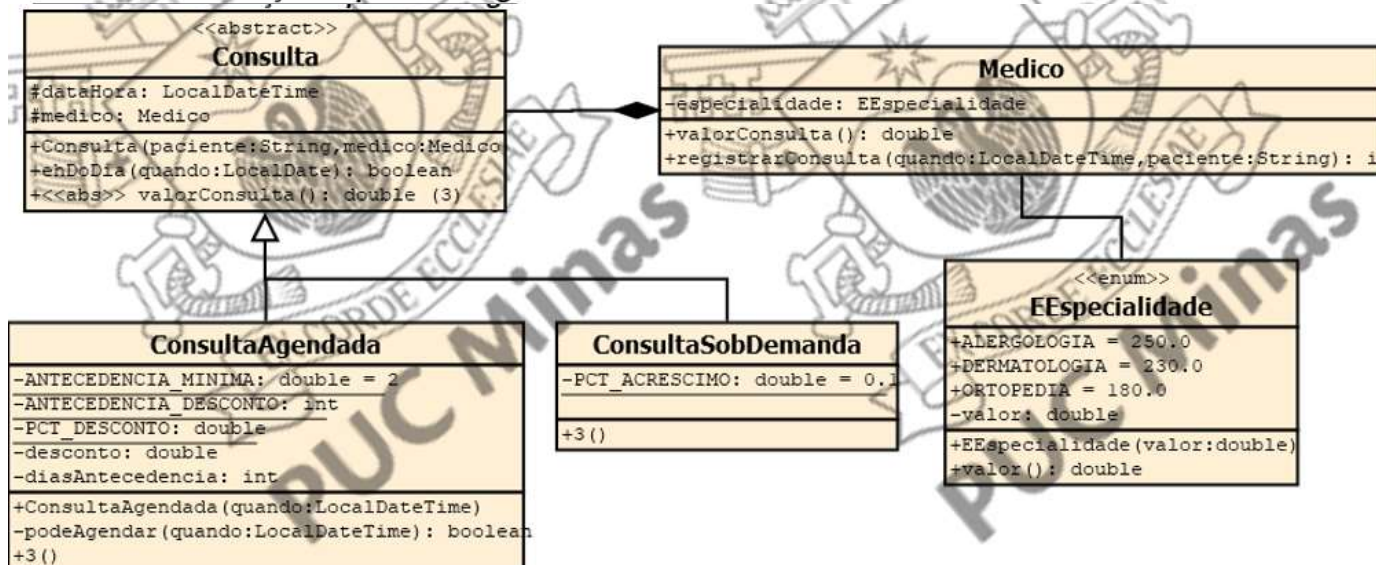
    StringBuilder agenda = new StringBuilder(
        String.format("Agenda do médico %s (%s)\n", nome ,registroCRM));
    for(Consulta consulta in consultas){
        if(consulta.getDia().equals(data))
            agenda.appendline(consulta+ "\n");
    }
    return agenda.toString();
}

```

2) O modelo proposto altera algumas coisas do modelo original. Em primeiro lugar, **especialidade** não pode ser armazenada como string no médico. As especialidades são limitadas e, mais importante, definem o valor da consulta. Assim, em lugar de uma string e outro atributo para o valor da consulta, seria mais indicado ter uma **enumerador para especialidades**, com **métodos** de extensão para representar esta responsabilidade. Em segundo lugar, se o método **"getDia"** da consulta está servindo somente para verificar o dia no momento da agenda, seria uma melhora na abstração **repassar esta "pergunta" para a classe Consulta, trocando-o pelo método "ehDoDia"**.

Consulta será uma classe **abstrata**, pois ela deve obrigatoriamente ser **Agendada** ou **SobDemanda**. Ela **armazena o médico** para poder saber sua especialidade. O método do **valor da consulta é abstrato** por ter regras diferentes em cada tipo.

Observe, no diagrama, a utilização das **"dicas" para responder uma questão de modelagem** na prova: **anotações** para abstrato e enumerador; **enumerador** "como se fosse uma classe" com seus métodos; **"numeração" do método** para não precisar reescrever a declaração no **override** e mostrando, para as classes **Medico** e **Consulta**, **somente as modificações a partir da original**.



```

3)
public double valorConsulta(){
    double valor = medico.valorConsulta();
    if(diasAntecedencia >= ANTECEDENCIA_DESCONTO)
        desconto = valor * PCT_DESCONTO;
    return valor - desconto;
}

```

Neste caso, o construtor precisa calcular "diasAntecedencia" e verificar a antecedência mínima:

```

public ConsultaAgendada(LocalDateTime quando, Medico medico, Paciente paciente){
    diasAntecedencia = DateTime.Now().until(quando, DAYS);
    if(diasAntecedencia < ANTECEDENCIA_MINIMA || medico == null || paciente == null)
        throw new IllegalArgumentException();

    super(quando, medico, paciente);
    dataHora = quando;
}

```

```

4)
public static Consulta criarConsultaAgendada(){
    Consulta nova = null;
    IO.PLN("Nova consulta agendada");

    String crm = IO.RLN("Digite o CRM do médico: ");
    Medico medico = (Medico)localizar(crm, listaMedicos);

    String id = IO.RLN("Digite a identidade do paciente: ");
    Paciente paciente = (Paciente)localizar(id, listaPacientes);

    String data = IO.RLN("Qual é a data da consulta? ");
    String hora = IO.RLN("Qual é a hora da consulta? ");
    LocalDateTime quando = LocalDateTime.parse(data + " " + hora);

    try{
        nova = new ConsultaAgendada(quando, medico, paciente);
    }catch (IllegalArgumentException ile){
        IO.PLN("Verifique a antecedência da consulta, dados do médico e paciente");
    }
    return nova;
}

```

Não era necessário na questão, mas o método "localizar" genérico seria:

```

private static Object localizar(String id, List lista){
    for (Object obj : lista) {
        if(obj.hashCode() == id.hashCode())
            return obj;
    }
    return null;
}

```